

Active Directory nesting groups strategy and implementation

 techtarget.com/searchwindowsserver/tip/Active-Directory-nesting-groups-strategy-and-implementation

Adam Bertram

April 8, 2020



Sergey Nivens - Fotolia

Does your current Active Directory permissions setup spark joy? If not, then it's time to unscramble that confusing design into something that's easier to use and maintain.

Trying to set up nesting groups in Active Directory can quickly become a challenge, especially if you don't have a solid blueprint in place.

Microsoft recommends that you apply a nesting and role-based access control ([RBAC](#)), specifically the AGDLP for single-domain environments and AGUDLP for multi-domain/multi-forest environments. But implementing either arrangement in a legacy setup that lacks a clear strategy when it comes to RBAC and nesting can take time to clean up. The effort will be worthwhile, because the end result will make your environment more secure and dynamic.

Why should I use a nesting groups strategy?

A good nesting approach, such as AGDLP or AGUDLP, gives you a great overview of who has [what permissions](#), which can help in certain situations such as audits. This setup is also useful because it eliminates the need for troubleshooting if something doesn't work. Lastly, it reduces administrative overhead by making the [assignment of permissions](#) to other domains straightforward.

What is AGDLP?

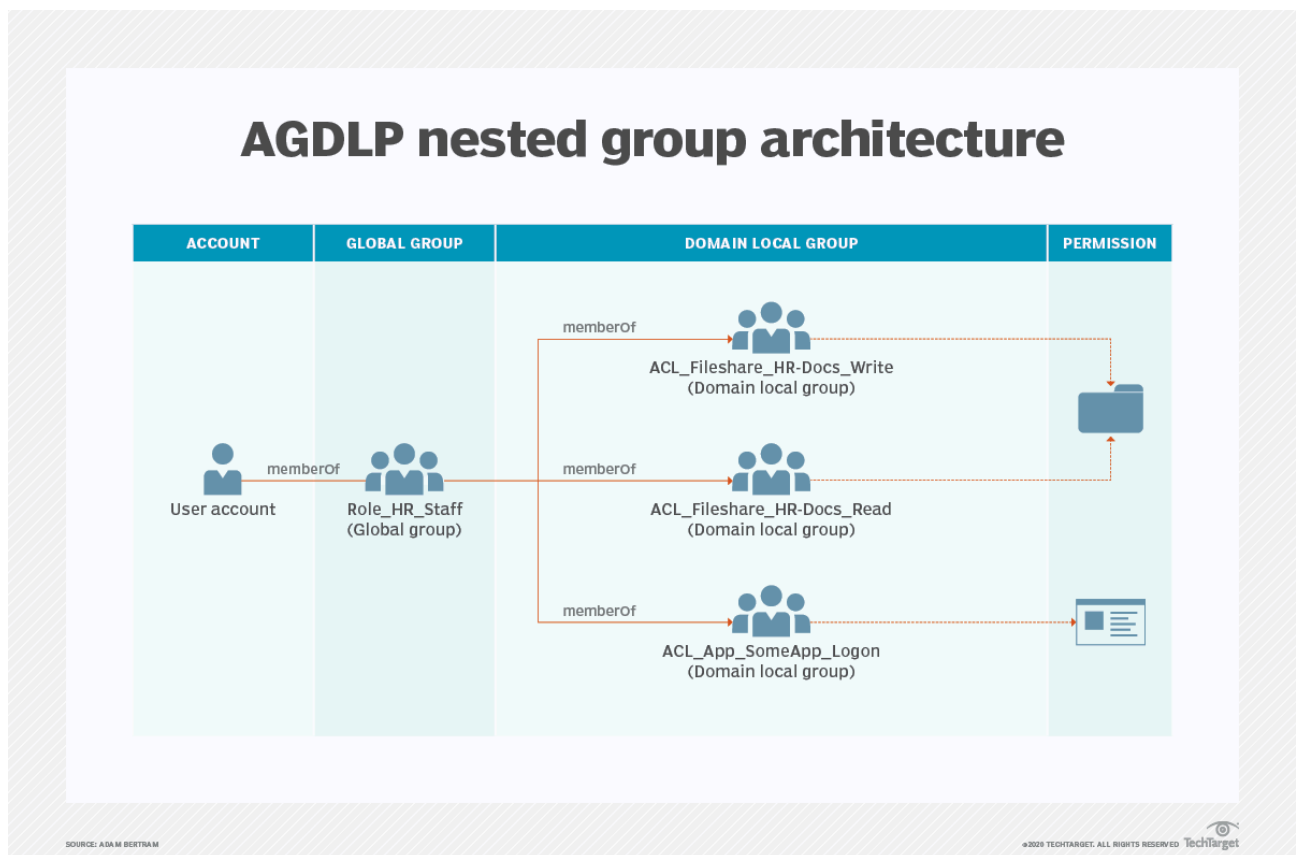
AGDLP stands for:

- **A**ccounts (the user or computer)
- **G**lobal group (also called role group)
- **D**omain **L**ocal groups (also called access groups)
- **P**ermissions (the specific permission tied to the domain local group)

The acronym is the exact order used to nest the groups.

Accounts will be a member of a global group that, in turn, is a member of a domain local group. The domain local group holds the specific permission to resources we want the global group to have access to, such as files and printer queues.

We can see in the illustration below how this particular nesting group comes together:



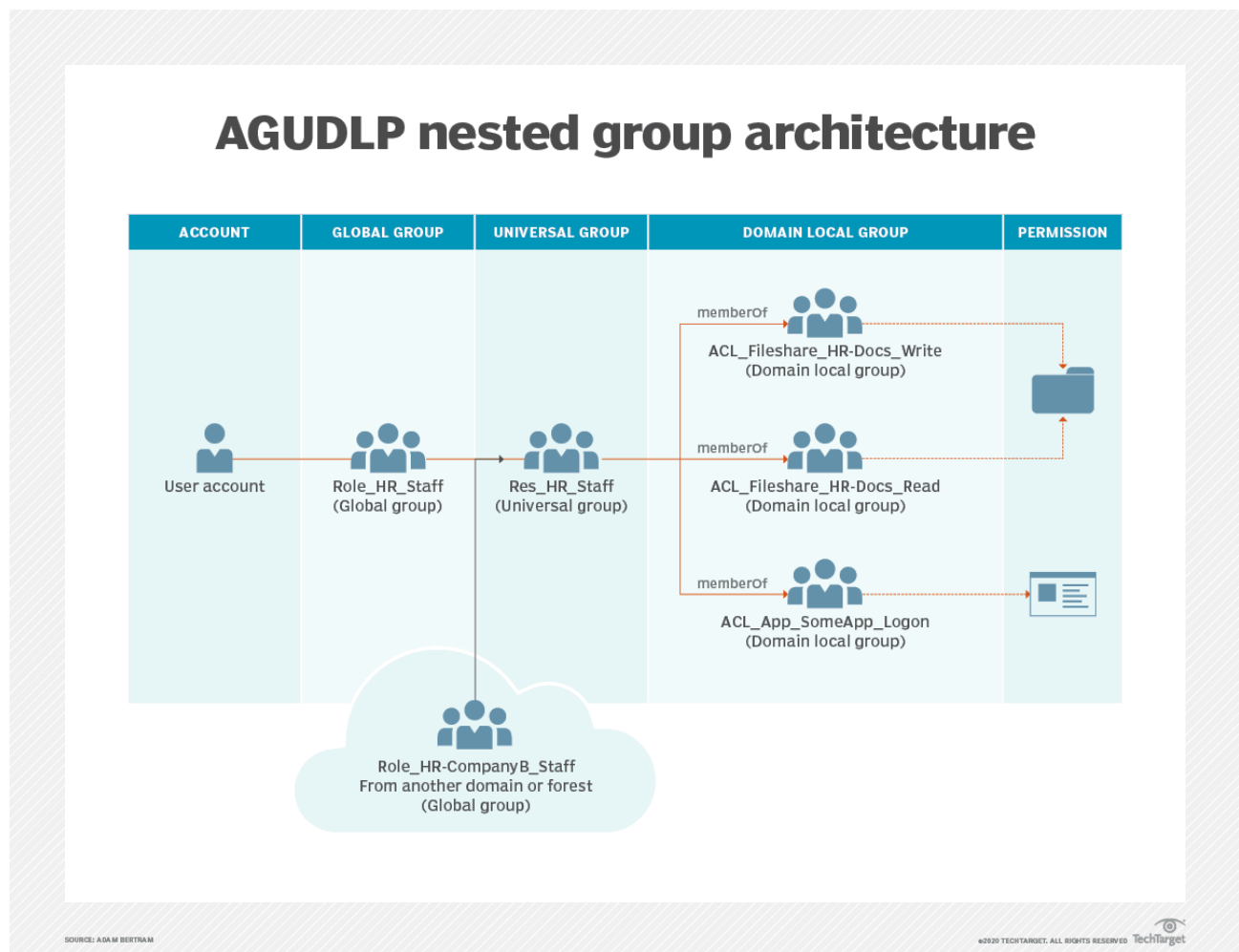
AGDLP is Microsoft's recommended nesting group for role-based access configuration in a single domain setting.

By using AGDLP nesting and RBAC principles, you get an overview of a role's specific permissions, which can be easily copied to other role groups if needed. With AGDLP, you only need to remember to always tie the permission to the domain local group at the end of the nesting chain and never to the global group.

What is AGUDLP?

AGUDLP is the multi-domain/multi-forest version of AGDLP, with the one difference being a universal group added to the nesting chain. You can use these universal groups to add role groups (global groups) from other domains without too much effort.

The universal group -- also called a resource group -- should have the same name as the corresponding role group, except for its prefix, as illustrated below:



For organizations with multiple domains and forests, AGUDLP is recommended to make it easier to add role groups from other domains.

What are the implementation concerns with AGDLP/AGUDLP?

There are four important rules related to the use of AGDLP or AGUDLP:

1. Decide [on a naming convention](#) of your groups.
2. One user can have multiple roles. Don't create more role groups than necessary.
3. Always use the correct group type: domain local, global, universal, etc.
4. Never assign permissions directly to the global or universal groups. This will break the nesting strategy and its corresponding permissions summary for the organization.

Should you use AGDLP or AGUDLP?

If you don't need to assign permissions across multiple domains, then always use AGDLP. Groups nested with AGDLP can be converted to AGUDLP if needed and require less work to operate. If you're in doubt, use AGDLP.

To convert an AGDLP nested group to AGUDLP, do the following:

1. Create a universal group.
2. Transfer the memberships of the global group to the universal group.
3. Add the universal group as a member of the global group.
4. Have all users and computers update their [Kerberos](#) ticket or log out and log in.
5. Remove all the domain local groups from the global group.

Why a naming convention is necessary with nesting groups

You should decide on a naming convention before you implement AGDLP or AGUDLP; it's not a requirement, but without one, you will quickly lose control of the organization you worked to build.

There are multiple naming schemes, but you can create a customized one that fits your organization. A good naming convention should have the following criteria:

- Be easy to read.
- Be simple enough to [parse with scripts](#).
- Contain no whitespace characters, such as spaces.
- Contain no special characters -- characters that are not numbers or from the alphabet -- except for the underscore or minus sign.

Here are a few examples for the different group types:

Role groups

Naming convention: Role_[Department]_[RoleName]

Examples: Role_IT_Helpdesk or Role_HR_Managers

If you use the AGUDLP principle, then there should be a corresponding resource group with a **Res** prefix such as Res_IT_Helpdesk or Res_HR_Managers.

Permission groups (domain local groups)

Naming convention: ACL_[PermissionCategory][*PermissionDescription*][PermissionType]

Examples: ACL_Fileshare_HR-Common_Read or ACL_Computer_Server1_Logon or ACL_Computer_Server1_LocalAdmin.

Executing AGDLP and AGUDLP

It might be challenging to implement AGDLP in older domains that lack a conventional arrangement. It's imperative to identify and test thoroughly to uncover a lot of the oddities to make everything conform to the new setup.

A rough outline of the implementation plan looks like this:

- Educate and inform your co-workers to keep them from creating groups and assigning permissions in a way that doesn't adhere to the new arrangement.
- Ask the HR department for assistance to identify roles. It's possible a user might have multiple roles.
- Create role groups and their corresponding **Res** groups -- if you use AGUDLP -- and assign new permissions with the AGDLP/AGUDLP principle.
- Identify existing permissions and [change them](#) to adhere to AGDLP/AGUDLP. You could either rename the groups and adjust their group type or build new groups side by side with the intent to replace the old group at a later date.